Smart Manufacturing Final Presentation

Smart Manufacturing Approach to Solving Problems in Injection Molding

202112460 Yuna Park

202560018 Gatienne

202214215 Gunhee Kim

202011402 Bomi Kim

LIST OF CONTENTS

01
Recap

[Recap] Roles in Our Project

Yuna Park

-Team Leader

-SOFTWARE :REINFORCEMENT LEARNING

-DESIGNING PPT

Gatie nne

-SOFTWARE :REINFORCEMENT LEARNING

-PRESENTATION

Gun hee Kim

-HARDWARE :RASPBERRY PI CIRCUIT, CODE

-DESIGNING PPT

Bomi Kim

-HARDWARE :RASPBERRY PI CIRCUIT, CODE

-DESIGNING PPT

[Recap] Problem Definition

1 Key parameters are adjusted by human

Lack of Consistency

No Continuous Optimization

Dependant on Skilled Workers

Time-Consuming Tuning

[Recap] Problem Definition

2 Low rates of yields

Waste of Material

Reduced Productivity

Increased Rework

Decreased Reliability

[Recap] Problem Definition

3 Hardships in standardization

Lack of Reproducibility

Difficulty in Data-Driven Quality Control

Inefficiency in Repeated Testing

Inapplicability to Smart Factory Systems

02

Problem Solution

Problem Solution

1. Analyzing Relationships between Key Parameters and Defect Rate
2. Calculating Optimal Range of Key Parameters
3. Simulating with Code
4. Visualizing MVP
5. Availability for SMEs to Improve Yield Rate without Cost Problem

MVP for Software

```
Rows : 7996
Labels: label
0    0.991121
1    0.008879
Name: proportion, dtype: float64
```

	count	mean	std	min	25%	50%	75%	max
PART_FACT_SERIAL	7996.0	11.248749	5.005191	2.00	9.00	10.000000	13.000000	24.000000
Injection_Time	7996.0	8.242189	3.107724	1.05	9.52	9.560000	9.590000	16.309999
Filling_Time	7996.0	3.896309	1.284451	0.93	4.40	4.440000	4.470000	8.270000
Plasticizing_Time	7996.0	16.209505	1.443720	10.92	16.58	16.799999	16.889999	21.120001
Cycle_Time	7996.0	59.900902	0.885680	58.84	59.52	59.520000	59.560001	66.860001

	cycle_time	filling_time	plasticizing_time	clamp_close_time	max_inj_pressure	max_back_pressure	avg_back_pressure	max_screw_rpm	avg_screw_rpm	max_
0	61.779999	0.93	12.910000	6.81	142.399994	55.500000	60.500000	30.900000	290.399994	1
1	59.459999	4.39	16.799999	7.12	141.800003	37.500000	59.299999	30.700001	29.200001	
2	59.480000	4.39	16.820000	7.13	141.699997	37.400002	59.200001	30.600000	29.200001	
3	59.439999	4.38	16.820000	7.12	141.699997	37.099998	59.099998	30.400000	29.200001	
4	59.520000	4.46	16.889999	7.13	142.000000	38.099998	59.599998	30.700001	29.200001	

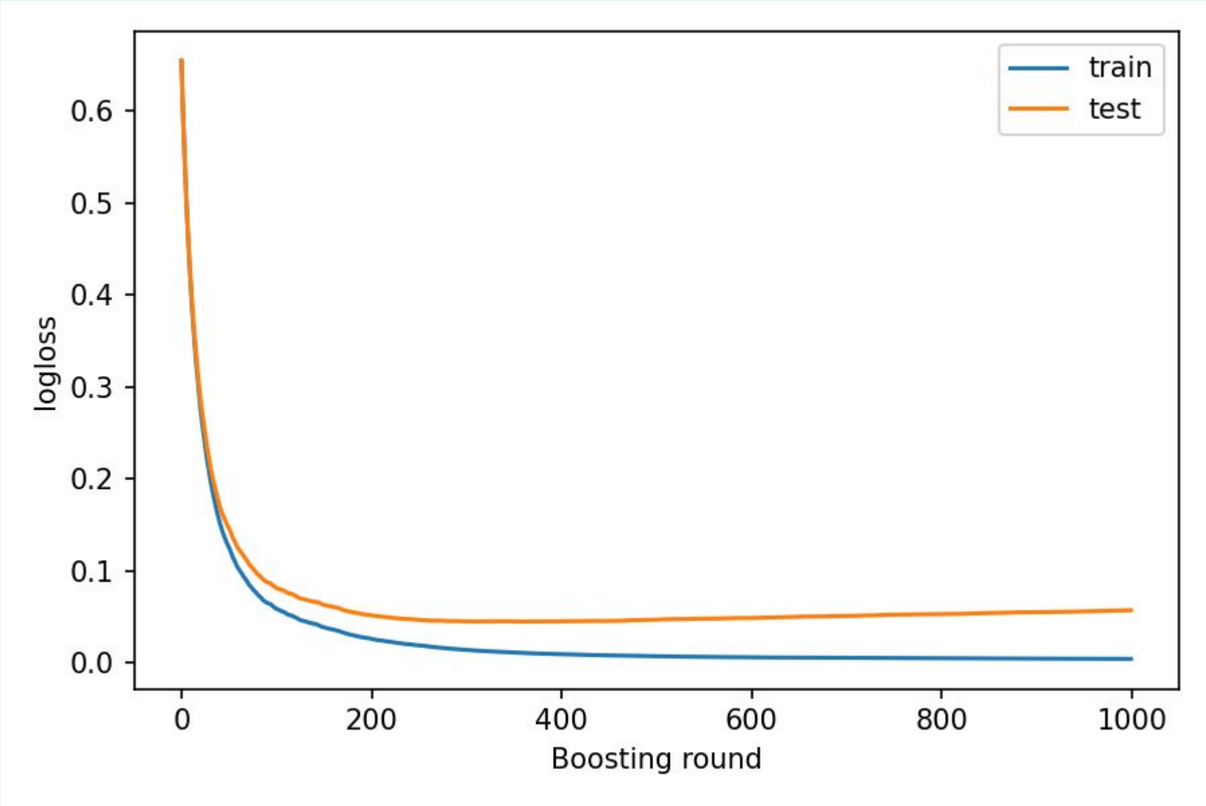
MVP for Software

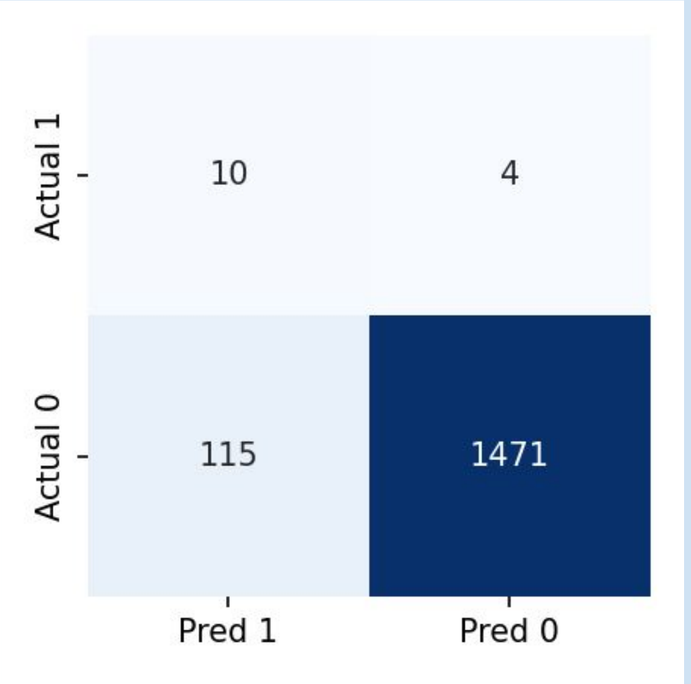
feature_plan.md > # Cycle-Level Feature Plan (Molding-QA dataset)

Cycle-Level Feature Plan (Molding-QA dataset)

Feature name (column)	Source column(s)	Rationale / what it captures
cycle_time	`Cycle_Time`	Overall productivity KPI
filling_time	`Filling_Time`	Long fill → short-shot risk
plasticizing_time	`Plasticizing_Time`	Material melt quality
clamp_close_time	`Clamp_Close_Time`	Slow clamp may cause flash
max_injection_pressure	`Max_Injection_Pressure`	Peak cavity/ram pressure → flash / stress
max_back_pressure	`Max_Back_Pressure`	Excess melt back pressure → overheating
avg_back_pressure	`Average_Back_Pressure`	Long-term drift indicator
max_screw_rpm	`Max_Screw_RPM`	Extreme RPM unstable material feed
avg_screw_rpm	`Average_Screw_RPM`	Melt homogeneity trend
max_injection_speed	`Max_Injection_Speed`	Fast + high pressure ⇒ flash; too slow ⇒ short-shot
cushion_error	`Cushion_Position` − target (5 mm)	Shot size consistency
switch_over_error	`Switch_Over_Position` − set-point (15 mm)	Fill/pack switchover accuracy
barrel_temp_avg	mean(`Barrel_Temperature_1…7`)	Material viscosity proxy
hopper_temp	`Hopper_Temperature`	Moisture control
mold_temp_avg	mean(`Mold_Temperature_1…12`)	Surface finish & cycle driver
delta_mold_temp	max − min of 12 mold temps	Gradient → warpage risk
outlier_flag	from `outlier_flag.py`	IQR anomaly indicator
label	`label` (0 = pass, 1 = fail)	Supervised target & RL penalty

MVP for Software





Precision = 0.08
Recall = 0.71
F1 = 0.14

MVP for Hardware

```python
import RPi.GPIO as GPIO
import time
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from PIL import Image, ImageDraw, ImageFont

# GPIO 설정
GPIO.setmode(GPIO.BCM)
LED_PINS = {'red': 17, 'green': 27, 'blue': 22}
for pin in LED_PINS.values():
    GPIO.setup(pin, GPIO.OUT)

# OLED 설정
serial = i2c(port=1, address=0x3C)
device = ssd1306(serial)
font = ImageFont.load_default()

# 임계값
TEMP_THRESHOLD = 50
PRESSURE_THRESHOLD = 1000  # hPa

# OLED 출력 함수
def display_status(temp, press, status):
    image = Image.new('1', device.size)
    draw = ImageDraw.Draw(image)
    draw.text((0, 0), f"T: {temp:.1f}C\nP: {press:.1f} hPa\n{status}", font=font, fill=255)
    device.display(image)

# LED 제어 함수
def set_led(color):
    for c in LED_PINS:
        GPIO.output(LED_PINS[c], GPIO.HIGH if c == color else GPIO.LOW)

```

✓ GPIO 17 → RED
✓ GPIO 22 → BLUE
✓ GPIO 27 → GREEN

- Use input functions to enter temperature and pressure values.

- Display the entered values and status on the OLED display.

- Clear the display each time new values are entered.

  Thresholds:
  Screw Velocity > 10℃
  Pressure > 10 hPa
  Cooling Time >

  Status:
  - FAULT + LED(RED) -> When Threshold is Exceeded
  - NORMAL + LED(BLUE) -> When Within Threshold

- Use a switch to reset/initialize the system.

MVP for Hardware

```
"""
Raspberry Pi Quality-Alarm Demo
GREEN = OK
RED   = NG Alarm  (prob_fail ≥ 0.0020)
BLUE  = Sensor fault (temp > 50 °C or pressure > 1000 hPa)
"""

# — add project root to sys.path so "src" imports work ————————————
import sys, pathlib
ROOT = pathlib.Path(file).resolve().parent  # repo root if script lives there
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import time, csv, numpy as np

# — ML wrapper (uses locked threshold) ————————————————————————
from src.models.xgb_quality_wrapper import prob_fail

# — Try GPIO & OLED; fall back to console if not on Pi ————————————
try:
    import RPi.GPIO as GPIO
    from luma.core.interface.serial import i2c
    from luma.oled.device import ssd1306
    from PIL import Image, ImageDraw, ImageFont
    HARDWARE = True
except (ImportError, RuntimeError):
    print("⚠ Running in console-only mode (no GPIO/I2C libs found)")
    HARDWARE = False

# — LED pins (BCM) - adjust to your wiring ————————————————————
LED_PINS = {"green": 5, "red": 17, "blue": 0}   # use three distinct pins
if HARDWARE:
    GPIO.setmode(GPIO.BCM)
    for pin in LED_PINS.values():
        GPIO.setup(pin, GPIO.OUT)
```

```
def set_led(color):
    if not HARDWARE:
        print(f"[LED] {color.upper()}")
        return
    for name, pin in LED_PINS.items():
        GPIO.output(pin, GPIO.HIGH if name == color else GPIO.LOW)

# — OLED helpers ——————————————————————————————————————————
if HARDWARE:
    serial = i2c(port=1, address=0x3C)
    oled   = ssd1306(serial)
    font   = ImageFont.load_default()

    def oled_show(lines):
        img  = Image.new("1", oled.size, "black")
        draw = ImageDraw.Draw(img)
        for i, txt in enumerate(lines):
            draw.text((0, i * 10), txt, font=font, fill=255)
        oled.display(img)
else:
    def oled_show(lines):
        for l in lines: print(l)
        print("-" * 25)

# — Limits for sensor-fault condition ————————————————————————
TEMP_LIMIT   = 50.0   # °C
PRESS_LIMIT  = 1000.0 # hPa

# — Feature column order (scaled values) ——————————————————————
FEATURE_ORDER = [
    "cycle_time", "filling_time", "plasticizing_time", "clamp_close_time",
    "max_inj_pressure", "max_back_pressure", "avg_back_pressure",
    "max_screw_rpm", "avg_screw_rpm", "max_inj_speed",
    "cushion_error", "switch_over_error",
    "barrel_temp_avg", "hopper_temp",
    "mold_temp_avg", "delta_mold_temp", "outlier_flag",
]
```

MVP for Hardware

```python
# ── Data source: CSV replay  (demo_50.csv must contain scaled columns) ──
CSV_FILE = ROOT / "demo_50.csv"
rows = csv.DictReader(open(CSV_FILE, newline=""))

# ── Main loop ──────────────────────────────────────────────
try:
    for row in rows:
        # build feature vector in correct order
        features = np.asarray([float(row[c]) for c in FEATURE_ORDER], dtype=np.float32)

        # temp & pressure for the OLED / sensor-fault rule
        temp  = float(row.get("barrel_temp_avg", 40.0))     # fallback dummy
        press = float(row.get("max_inj_pressure", 800.0))   # fallback dummy

        # 1) sensor-fault?
        if temp > TEMP_LIMIT or press > PRESS_LIMIT:
            set_led("blue")
            oled_show([f"T={temp:.1f}°C", f"P={press:.0f} hPa", "SENSOR FAULT"])
            time.sleep(2)
            continue

        # 2) ML probability
        p_fail = prob_fail(features)
        if p_fail >= 0.0020:
            set_led("red")
            status = f"ALARM  p={p_fail:.3f}"
        else:
            set_led("green")
            status = f"OK     p={p_fail:.3f}"

        # 3) OLED lines
        oled_show([f"T={temp:.1f}°C", f"P={press:.0f} hPa", status])
        time.sleep(2)

except KeyboardInterrupt:
    print("Stopped by user")
```

```python
finally:
    if HARDWARE:
        oled.clear()
        GPIO.cleanup()
```

✓ GPIO 17 → RED
✓ GPIO 22 → BLUE
✓ GPIO 27 → GREEN

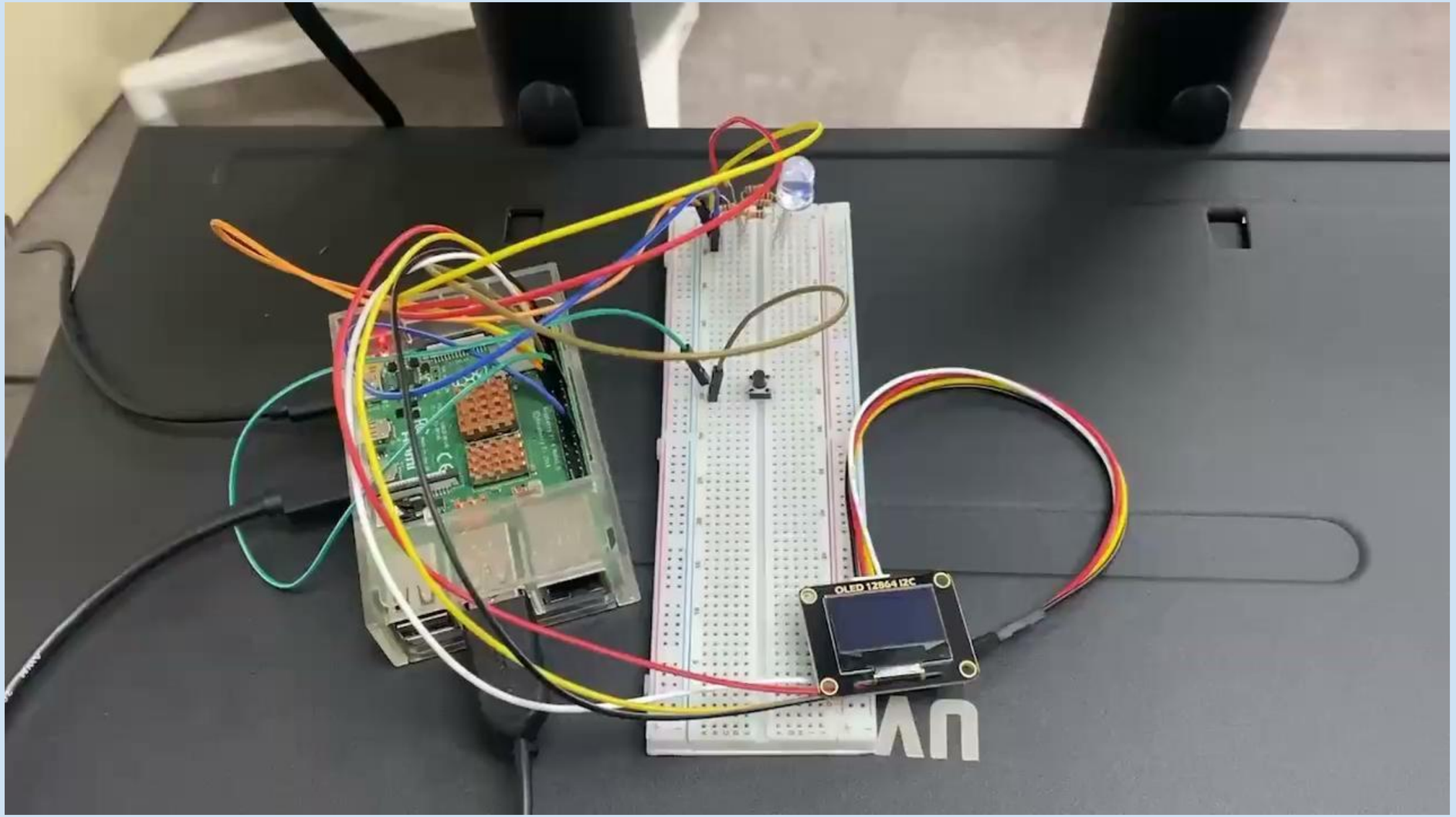
- Read the dataset from csv
- Display the data, status and P-value on the OLED display.
- Clear the display each time there is a new value

Thresholds: 0.002

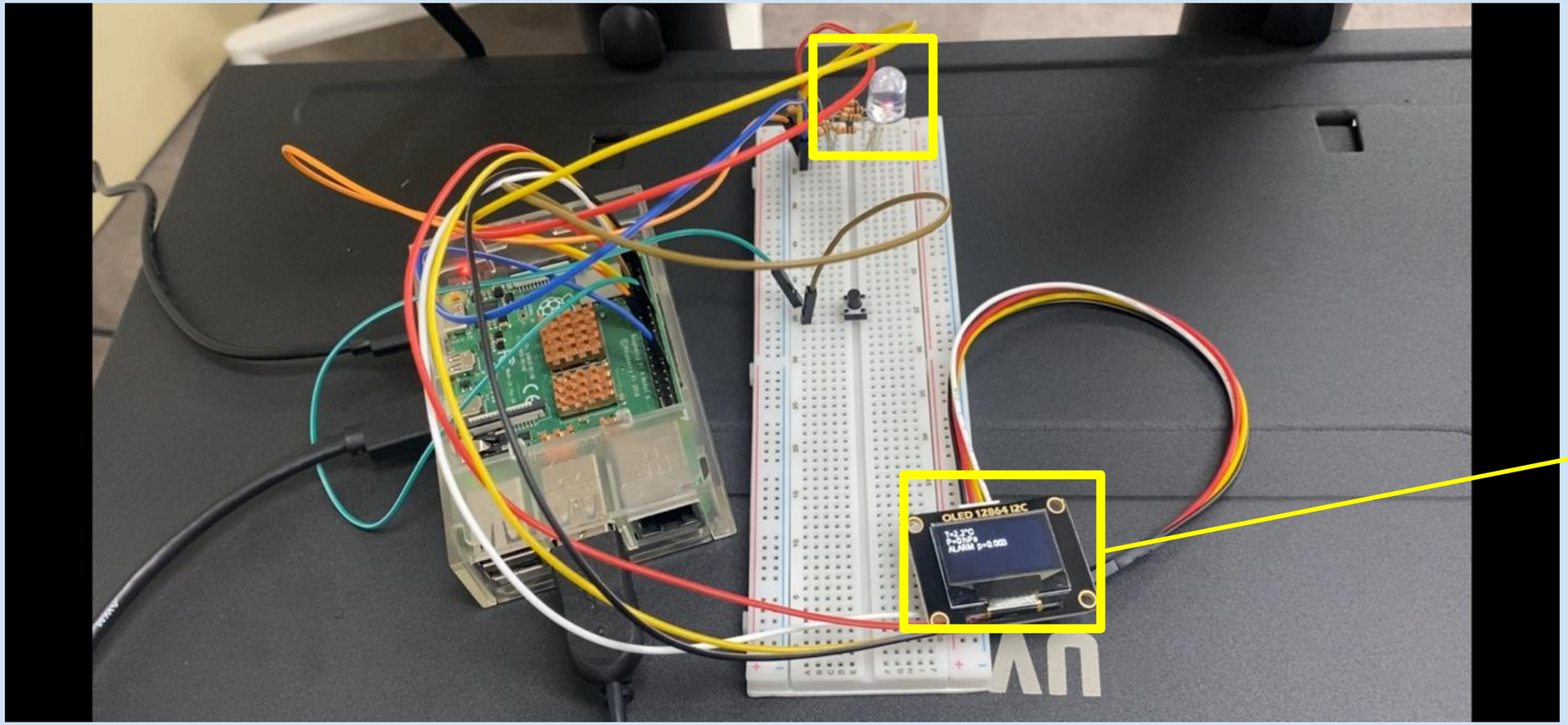
Status:
- ALARM + LED(RED) -> P-value > 0.002
- OK + LED(BLUE) -> P-value <= 0.002

03
Demonstration

Demonstration



Demonstration



T=2.2℃
P=0 hPa
ALARM p=0.003

04

Discussion

Discussion

Comparison

Aspect	KAIST Guide-book Study (PDF, KR)	johnwslee GitHub Repo	Our Project (Molder-RL)
Purpose	Offline benchmark set for defect-classification (RF, DNN, VAE)	Fully reproducible tutorial on basic anomaly detection	Real-time quality alarm & control MVP with LED/PLC interaction
Data Slices	Two mould sets: CN7 & RG3 (44 raw features each)	Same sets with minimal cleaning (26 features)	CN7 only (18 features) → Plan to extend to RG3
Imbalance Handling	Oversampling + class weights (defect ≈ 15%)	None (defect ≈ 1%) → low precision	Threshold tuning + real-time SMOTE / cost-sensitive XGB.
Output Latency	Offline scoring via CSV for research	Offline Jupyter Notebook demo	**< 1 s alarm** + optional control signal output to hardware
Explainability	None (only tables in PDF)	SHAP plots for educational use	Streamlit dashboard with live highlight and LED push for operator visibility

Discussion

Limitations

1. Insufficient Normal / Abnormal data
2. Low accuracy and limited range of low-cost sensors
3. Vulnerability to environmental noise and interference
4. Absense of real-time industrial-grade detection sensor

Discussion

Developments in the future

1. Building a real-time monitoring system
2. Collecting industrial-range data for training
3. Improve durability and defect classification

THANKS
FOR LISTENING